



Herald College, Kathmandu

Artificial Intelligence and Machine Learning (6CS012)

Part I – Question and Answer Report

Group Name: STChange9

Members:

Suyog Bastakoti (2407093)

Bhuwan Baniya (2414002)

Sujen Tamang (230133)

Submitted To: Jinu Nachhyon

Submitted by: Bhuwan Baniya

5/10/2026

ID: 2414002

Long Question Answer (5 Marks)

Question 2: ML Engineering Challenges in Production

The following answer addresses the key challenges encountered when deploying and maintaining ML systems in production, with practical solutions and a reflection on cross-functional collaboration.

Challenge 1: Data Drift

Data drift occurs when the statistical properties of the input data change over time, causing the model to make increasingly inaccurate predictions. For example, a fraud detection model trained on 2023 transaction patterns may become unreliable in 2025 as spending habits and attack vectors evolve. If left unaddressed, the model silently degrades — accuracy may drop by 10–20% before anyone notices. The practical solution is to implement continuous monitoring of input feature distributions using tools such as Evidently AI or WhyLogs, combined with automated retraining pipelines triggered when drift exceeds a defined threshold. The trade-off is cost: frequent retraining consumes compute resources and may require re-labelling new data, which is expensive and time-consuming.

Challenge 2: Model Degradation and Retraining Latency

Even without data drift, model performance degrades as the real world evolves in ways not captured in the training set. An e-commerce recommendation engine trained before a major product line expansion will miss newly popular categories entirely. If degradation is not caught promptly, user engagement drops and revenue is impacted. A shadow deployment strategy — where a newly retrained model runs in parallel with the live model before being promoted — allows safe validation without risking user experience. MLOps platforms such as MLflow and Kubeflow automate the retraining and deployment workflow. The trade-off here is engineering complexity: maintaining parallel serving infrastructure requires dedicated DevOps effort and increases infrastructure costs.

Challenge 3: Latency and Scalability

Large models like deep neural networks or ensembles can be too slow for real-time serving. An e-commerce platform that takes three seconds to return product recommendations will lose users to competitors. High-traffic events such as flash sales can also overwhelm serving infrastructure not built for elastic load. Solutions include model quantisation and pruning to reduce inference time, horizontal autoscaling using Kubernetes, and caching frequent predictions. LLM-based systems specifically benefit

from speculative decoding and KV cache optimisation. The trade-off is accuracy versus speed: quantised models are faster but may sacrifice a few percentage points of performance.

Cross-Functional Collaboration

Reliable ML systems require close coordination across data scientists, engineers, and product managers. Data scientists design experiments and monitor model quality; engineers own the deployment pipeline, API reliability, and infrastructure scaling; product managers define acceptable performance thresholds and prioritise which models to update first. Without this alignment, a technically excellent model can fail in production because of poor API design, misaligned business metrics, or insufficient compute provisioning. Shared dashboards, defined SLAs for model performance, and regular cross-team reviews are practical mechanisms that bridge these disciplines.

Role of Modern Tooling

LLM-based systems and automation pipelines are reducing manual overhead in the ML lifecycle. Tools such as automated feature engineering platforms, self-healing pipelines, and AI-assisted code review can flag data quality issues, suggest hyperparameter adjustments, and generate documentation — allowing engineers to focus on higher-level architecture decisions rather than repetitive operational tasks.

Short Question – Section 1: Overfitting (2.5 Marks)

Question 2: Techniques to Reduce Overfitting

Overfitting happens when a model learns the training data too precisely, including noise, and fails to generalise to unseen examples. Two widely used regularisation techniques address this:

Dropout

Dropout randomly deactivates a proportion of neurons during each training step. The intuition is that the network cannot rely on any single neuron, forcing it to develop multiple independent pathways to reach the same prediction. This prevents co-adaptation and acts as an implicit ensemble of many smaller networks. In practice, a dropout rate of 0.2 to 0.5 after dense or recurrent layers is common. In an image classification task such as detecting pneumonia in chest X-rays, dropout after convolutional layers prevents the model from overfitting to the specific appearance of artefacts in the training scans. During

inference, all neurons are active and their outputs are scaled by the dropout probability to maintain expected values.

Early Stopping

Early stopping monitors validation loss at the end of each epoch and halts training as soon as it begins to rise, then restores the weights from the epoch where validation loss was lowest. The intuition is straightforward: the point of minimum validation loss is where the model has learned genuine patterns without yet memorising training noise. In a text classification task such as sentiment analysis on tweets, training may converge after four epochs, but continuing to epoch fifteen simply causes the model to memorise specific phrases from the training set. Early stopping with the patience of three epochs avoids this without requiring any modification to the model architecture.

Short Question – Section 2: Neural Network Architecture (2.5 Marks)

Question 1: CNN vs RNN

Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) are designed for fundamentally different types of data and solve different structural problems.

Input Structure and Data Type

CNNs are designed for spatially structured data where local patterns are meaningful regardless of their position — primarily images, but also spectrograms and spatial sensor grids. A 224×224 pixel image is passed as a fixed-size grid. RNNs are designed for sequential data where the order of elements carries meaning — text, time-series, audio waveforms, or sensor readings over time. Each element is processed one step at a time.

Information Flow and Architecture

CNNs process inputs in a single forward pass through a stack of convolutional and pooling layers, extracting hierarchical spatial features. RNNs maintain a hidden state that is updated at each timestep, allowing information to flow from earlier positions in the sequence to influence later ones. This recurrence is what allows RNNs to model temporal dependencies — but it also introduces the vanishing gradient problem, where gradients shrink to near-zero as they propagate back through many timesteps.

Parameter Sharing

In CNNs, a single filter (kernel) is convolved across the entire spatial extent of the input, so the same weights detect the same local feature anywhere in the image. In RNNs, the same weight matrix is applied at every timestep, encoding the assumption that the relationship between inputs at each position follows the same transformation.

Real-World Examples

CNNs are the standard choice for image classification (e.g., recognising handwritten digits or medical imaging), object detection, and video frame analysis. RNNs, and specifically LSTMs, are preferred for machine translation, speech recognition, sentiment analysis of text, and stock price forecasting.

Training Challenges and Mitigations

The vanishing gradient problem affects deep CNNs and RNNs: gradients used to update early layers become negligibly small during backpropagation, preventing learning. For CNNs, Batch Normalisation normalises layer activations to stabilise training. For RNNs, the LSTM architecture introduces gating mechanisms (forget, input, output gates) that allow gradients to flow over longer sequences without vanishing. Exploding gradients — where gradients grow uncontrollably — are addressed in RNNs using gradient clipping, which caps the norm of the gradient vector before each update. Overfitting in both architectures is mitigated by dropout and early stopping, as described above.

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.